

Day 13 lab: a monitored inference application

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 5, 2026

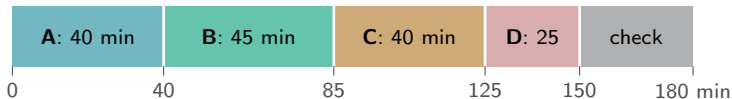
- ▶ **Goal:** a monitored inference application. The Day 8 detector measures itself, writes a log, and sends its metrics to a live dashboard. An alert rule tells you about drift.
- ▶ **Board:** the Raspberry Pi 5 with the camera.
- ▶ **Deliverable:** the live dashboard during a drift event, and the file `report.md` with the measured values, a short incident report, and the Decision Log.

From the lecture to the lab

The lecture gave the metrics of a device and of a model, structured logs with rotation, thresholds from a history, and an alert rule with a window and a duration. Its values came from stored images and a simulation. Today the camera sees your scene, and you change the light yourself.

Key point: Record a normal reference first. A threshold has a meaning only against your own normal values.

Plan of the three hours



Part	Work	Time
A	Instrument: measurements, a summary each 10 s, a JSON log with rotation	40 min
B	Dashboard: the summaries with MQTT, the live dashboard, a reference period	45 min
C	Drift: a threshold from the reference, a dim light, an unknown object	40 min
D	Alert: one rule with a window, a duration, and hysteresis	25 min
Check by the instructor		30 min

Write each result in `report.md` when you get it.

Hardware for each group

- ▶ Raspberry Pi 5 with the cooler, the camera, the card, and the 27 W power supply.
- ▶ Laptop in the lab network.
- ▶ A cup, a bottle, and an object that the model does not know.
- ▶ A desk lamp, or the light of the room.
- ▶ Optional: the XIAO with the Day 12 sketch.

Lab files

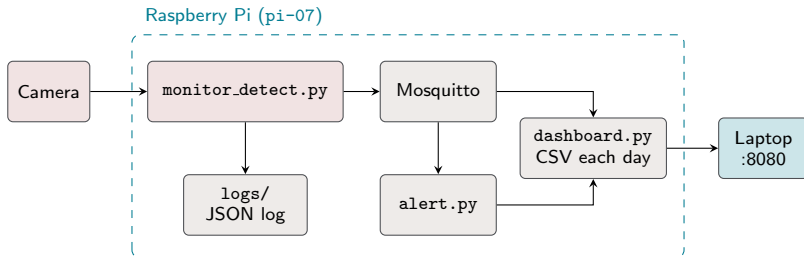
Labs/day13/ on the laptop, and a copy in
~/edgeai/day13/ on the Raspberry Pi.

Names

gNN is your group and pi-NN its board. The examples use g07.

Software

- ▶ Card: the environment ~/yolo (ultralytics, ncnn, paho-mqtt, psutil, flask) and the Mosquitto broker of Day 12.
- ▶ The Day 8 folder in ~/edgeai/day08/ with its model cupbottle_320_ncnn_model.
- ▶ Optional on the laptop: mosquitto_sub.



- ▶ `monitor_detect.py` runs the Day 8 detector. Each 10 s it writes one summary to the log and sends it to `edgeai/g07/pi/metrics`.
- ▶ `dashboard.py` stores each summary and draws eight charts. `alert.py` applies one rule and publishes its state on `edgeai/g07/pi/alert/<metric>`.

The summary of each 10 s

Field	Content
frames, fps	Frames in the window, frames each second
latency_ms, latency_p95_ms, latency_max_ms	Time of one frame: mean, 95th percentile, largest
confidence	Mean top score: the largest box score of each frame, 0 with no box
low_conf	Part of the frames with a top score below 0.5
objects	Mean number of boxes with a score of 0.25 or more
brightness, sharpness	Mean of the grey image, variance of its Laplacian (Task A1)
cpu_temp_c, throttled	Temperature, the bits of vcgencmd get_throttled
rss_mb, ram_used_mb, disk_free_pct	Memory of the detector, of the board; free disk

The model runs with the score threshold 0.05, so that the top score is known also when the model is not sure. The message also has the fields of HW-08: seq, ts, cpu_percent.

Part A: Tasks A1 and A2 (20 min)

Step	Work	Time
1	Copy the lab folder, check the Day 8 folder and its model	5 min
2	Tasks A1 and A2 in pi/monitor.py	15 min

```
scp -r day13 edge@pi-07.local:edgeai/ # laptop
ls ~/edgeai/day08/pi/detector.py ~/edgeai/day08/models/cupbottle_320_ncnn_model
cd ~/edgeai/day13/pi && ~/yolo/bin/python monitor.py # Pi
```

- ▶ **Task A1** `input_stats(frame)`: the grey image with `cv2.cvtColor`, its mean (brightness), and the variance of `cv2.Laplacian` (sharpness).
- ▶ **Task A2** `summarize(records, seconds)`: the ten keys of the docstring. The p95 uses `percentile()`, the nearest-rank rule of Day 9.
- ▶ **You see:** Task A1: complete and Task A2: complete.

Part A: run the detector, read the log (20 min)

Put the cup and the bottle in front of the camera.

```
~/yolo/bin/python monitor_detect.py \  
  --model ../../day08/models/cupbottle_320_ncnn_model \  
  --group g07 --no-mqtt --seconds 120  
wc -l -c ../logs/detector.jsonl  
tail -n 1 ../logs/detector.jsonl | python3 -m json.tool
```

You see one line each 10 s (here in two parts):

```
10:21:40 ... frames/s latency ... ms confidence ... low ... objects ...  
brightness ... sharpness ... .. C rss ... MB
```

- ▶ The log has one JSON record on each line: start, one summary each 10 s, stop. It has at most 1 MB and 5 old files.
- ▶ Cover the lens for 10 s: which values change?
- ▶ **You record:** one line, the records and the bytes of the log, and the WARNING records of a run with `--slow-ms 50 (grep -c slow_frame)`.

Source: The values of the lines come from your board.


```
systemctl is-active mosquito           # active (Day 12)
cd ~/edgeai/day13/dashboard
~/yolo/bin/python dashboard.py --group g07 # second SSH terminal
cd ~/edgeai/day13/pi                   # first terminal
~/yolo/bin/python monitor_detect.py \
    --model ../../day08/models/cupbottle_320_ncnn_model --group g07
```

- ▶ Open `http://pi-07.local:8080` on the laptop. The state bar says Waiting for data ..., then no alert program.
- ▶ The eight charts get one point each 10 s: frame rate, latency p95, temperature, memory of the detector, mean top score, part below 0.5, objects, brightness.
- ▶ On the laptop: `mosquitto_sub -h pi-07.local -t 'edgeai/g07/pi/#' -v`.
- ▶ **You record:** the messages and the lost messages of the state bar after 5 minutes, and the name of the CSV file in `dashboard/metrics_log/`.

Part B: reference period and Task C1 (20 min)



- ▶ Keep the scene normal for 10 minutes: the cup and the bottle in view, the normal light. People can move as usual. Do not stop the detector.
- ▶ Write the start and the end time. Write which panels move and which stay flat.
- ▶ During this time, complete **Task C1** `threshold_from_history(values, window, k)` in `pi/reference.py`: the means of 6 summaries in a row, their mean and standard deviation, the threshold mean $- 3$ sd, and the clear value mean $- 1$ sd.

Key point: The reference must show the normal variation of your scene. A still scene with no variation gives a threshold that fires too often.

Part C: a threshold from the reference (10 min)

```
~/yolo/bin/python reference.py \  
  --csv ../dashboard/metrics_log/metrics_2026-10-05.csv \  
  --start 10:20 --end 10:30 --group g07
```

You see:

```
Task C1: complete  
Reference: ... rows from 10:20:00 to 10:30:00, metric confidence  
Rows:      mean ... sd ... min ... max ...  
Means of 6 rows: ... means, mean ... sd ...  
Threshold: ... clear: ... (mean - ..., mean - ...)  
Command: python alert.py --group g07 --metric confidence --threshold ...
```

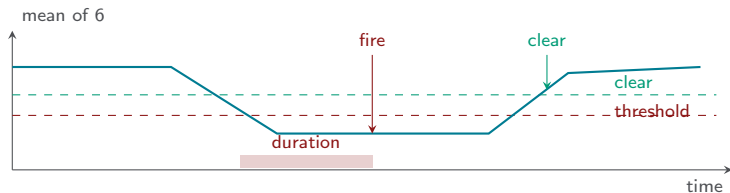
- ▶ Use the date of today and the times of your reference.
- ▶ A WARNING: the reference has almost no variation: run it again with `--min-margin 0.05`.
- ▶ Run it also with `--metric brightness`. **You record:** the mean, the standard deviation, the threshold, and the clear value.

Make each event for 3 minutes, with 3 minutes of the normal scene between them. Write the start and the end time.

Event	What you do	Watch
Dim light	Switch off the light of the room, or turn the lamp away	Mean top score, objects, brightness
Unknown object	Put your unknown object in place of the cup	Mean top score, part below 0.5, objects

- ▶ **You record:** for each event, the mean top score, the part below 0.5, the objects, the brightness, and the sharpness of the printed lines (the table of the report).
- ▶ Which metric shows each event most clearly? Which metric names the cause? The model gives no error in either event.
- ▶ With no lamp to switch: `--dim-after 60 --dim-gain 0.1` multiplies each frame by 0.1 for 3 minutes.

Part D: Task D1, the alert rule (10 min)



- ▶ **Task D1** `AlertRule.update(t, value)` in `pi/alert.py`: append the value; the mean of the last 6 values (60 s); fire when the mean stays below the threshold for the duration; clear when the mean is above the clear value.
- ▶ The clear value is above the threshold (hysteresis), so a mean near the threshold does not switch the alert on and off.
- ▶ The docstring gives the five steps. The program checks the method with a known sequence before it connects.

```
~/yolo/bin/python alert.py --group g07 --metric confidence \  
--threshold ... --clear ... --window 6           # the command of C1
```

You see during the dim light event:

```
Task D1: complete  
Rule: mean of 6 summaries of confidence below ... for 60 s; clear above ...  
HH:MM:SS ALERT ON      confidence: mean of 60 s ..., threshold ..., clear ...  
HH:MM:SS ALERT OFF     confidence: mean of 60 s ..., threshold ..., clear ...
```

- ▶ The state bar of the dashboard is red during the alert. With the XIAO of Day 12, --xiao-led switches its LED.
- ▶ A second copy with --metric brightness and its own threshold can watch the light.
- ▶ **You record:** the seconds from the start of the event to ALERT ON, and from its end to ALERT OFF. Then write the incident report.

The instructor checks this at the end of the lab:

- ▶ ✓ `monitor_detect.py` prints Task A1: complete and Task A2: complete, and the log has start, summary, and stop records.
- ▶ ✓ The dashboard shows the live system and model metrics, and the state bar shows no lost message.
- ▶ ✓ `reference.py` prints Task C1: complete and a threshold from your reference period.
- ▶ ✓ Live demonstration: during a dim light event, `alert.py` (with Task D1: complete) prints ALERT ON, and the state bar becomes red. After the event, the alert clears.
- ▶ ✓ The report has the incident report and a Decision Log with numbers and one trade-off.

Hand in: the file `report.md` with the drift table and the incident report.

A shop uses 20 Raspberry Pi cameras to count the people at its doors. One camera stands near a window, so its light changes with the time of day. The shop has one person who looks at alerts.

- ▶ Which metrics does each device send, how often, and where are they stored?
- ▶ Which alert rules do you use, with which thresholds, windows, and durations?
- ▶ How many false alerts each week do you expect, and what does the person do for each alert?

A good Decision Log (about 100 words):

- ▶ gives your measured size of a summary, your threshold, and the delay of your alert,
- ▶ explains the camera near the window: one reference is not enough,
- ▶ names what you lose: for example a fast alert against fewer false alerts.

If a part does not work, and common problems

Problem	Fix or fallback
the Day 8 file pi/detector.py is not found	Copy the Day 8 folder to ~/edgeai/day08/
The Day 8 model is not on the card	Download yolo11n.pt once, then --model yolo11n.pt --imgsz 320
The charts stay empty	Start the detector with --group g07 and no --no-mqtt
The state bar is yellow	No summary for 25 s: look at the terminal of the detector
The alert fires in the normal scene	A longer reference, or --min-margin 0.05
The alert does not fire during the event	An event longer than 2 minutes, or --for-seconds 30
The light of the room cannot change	--dim-after 60 --dim-gain 0.1 --dim-seconds 180
A task is not complete in time	Use the file of solutions/pi/, and write this in the report

The README of the lab has the complete list.

- ▶ *Machine Learning Systems* by Vijay Janapa Reddi and contributors, Harvard University. mlsysbook.ai, CC BY-NC-SA 4.0.
- ▶ *Edge AI Engineering: Raspberry Pi* by Marcelo Rovai. github.com/Mjrovai/EdgeML_Made_Ease_ebook and github.com/Mjrovai/EdgeML-with-Raspberry-Pi, GPL-3.0.